

COMPLETE FEATURES & TECHNICAL DOCUMENTATION

ESP32 16-Channel Automatic Relay Timer Switch

Every Feature · Every API · Every Setting

User Guide · Technical Reference · No Skips



Author: Raff Alds

GitHub: github.com/xiv3r · License: GPLv3

Firmware Config: EEPROM v11 · ExtConfig 0xEC

Source: `esp32-final-sketch.ino` (~3,982 lines)

Single-file Arduino sketch · Web UI · REST API · NVS · DS3231 · NTP

16 Relay Channels	8 Schedules per Relay	6 Web UI Pages	30+ REST API Endpoints	4 Time Sources	5 NVS Keys
--------------------------------	------------------------------------	-----------------------------	-------------------------------------	-----------------------------	-------------------------

This document is the **complete features catalog and technical documentation** for the ESP32 16-Channel Automatic Relay Timer Switch firmware. It covers every user-facing feature, every configuration option, every REST endpoint, every web page, every time source, scheduling rule, recovery mechanism, and hardware interface — with **nothing omitted**.

Table of Contents

1. Product Overview
2. Complete Feature Catalog (Master Checklist)
3. Hardware Requirements & Pinout
4. Getting Started / First Boot
5. Web User Interface — All 6 Pages
6. Relay Control Features
7. Scheduling Engine (Full Rules)
8. GPIO & Polarity Management
9. WiFi Station & Access Point
10. Timekeeping System (NTP / Browser / DS3231 / Internal RTC)
11. mDNS Discovery
12. Captive Portal
13. Self-Healing & Reliability
14. Factory Reset (Web + Hardware)
15. System Information & Diagnostics
16. Complete REST API Reference
17. Configuration & NVS Persistence
18. Default Values Reference
19. Runtime Architecture
20. Timing & Interval Reference
21. Security & Trust Model
22. Edge Cases & Error Handling
23. Libraries & Build Dependencies
24. Complete Function Index
25. Quick Reference Cards

1. Product Overview

The **ESP32 16-Channel Automatic Relay Timer Switch** is a production-oriented, single-file Arduino firmware that turns an ESP32 into a full-featured multi-channel timer controller with a built-in web configuration interface. It is designed for lighting, pumps, HVAC zones, irrigation, aquarium equipment, industrial timers, and any load that needs reliable scheduled switching — with or without internet.

1.1 What It Does

- Controls up to **16 independent relay channels** on configurable GPIOs
- Each channel supports **8 independent timer schedules** with second precision
- Schedules filter by **day-of-week**, **day-of-month**, and **month-of-year**
- Supports **overnight** (cross-midnight) windows and **always-ON** mode
- Manual ON/OFF override with one-tap return to automatic schedule mode
- Custom names per relay (double-click rename in the UI)
- Serves a complete mobile-friendly web UI over WiFi AP and/or Station
- Keeps accurate time via NTP, browser sync, DS3231 RTC, or software clock
- Self-heals WiFi, mDNS, AP, and relay pin states without rebooting
- Persists all settings in ESP32 NVS flash across power cycles

1.2 Design Principles

- **Works offline:** AP-only mode + DS3231/internal RTC — schedules run without internet
- **Safe boot:** All relays start OFF; manual states can be restored from checkpoint
- **No reboot required:** Most config changes (WiFi, AP, GPIO, schedules, polarity) apply live
- **Flash-friendly:** Critical state checkpointed on a throttle; RTC autosave only when offline
- **Millis-safe:** All timers survive the ~49.7-day millis() wrap

1.3 Identity

Item	Value / Description
Project title	ESP32 16-Channel Automatic Relay Timer Switch
Author	Raff Alds
GitHub	https://www.github.com/xiv3r
License	GPLv3
Config schema version	EEPROM_VERSION 11
SystemConfig magic	0x1234
ExtConfig magic	0xEC
GPIO config magic	0xD002
Critical state magic	0xDEADBEEF
NVS namespace	relay16
Source size	~3,982 lines (single .ino)
Chip target	ESP32 (documented as ESP32-38P)
Web server port	80
DNS captive port	53
mDNS service	_http._tcp port 80

2. Complete Feature Catalog (Master Checklist)

This is the authoritative inventory of every feature in the firmware. Use it for acceptance testing, sales specs, or onboarding.

A. Relay Channels & Output Control

- ✓ Up to 16 independent relay channels (runtime count 0–16)
- ✓ Dynamic GPIO pin assignment (add / remove / reset-to-default)
- ✓ Default pin map: 32, 33, 25, 26, 27, 14, 13, 23, 1, 3, 19, 18, 5, 4, 2, 15
- ✓ Logical ON/OFF abstraction independent of wiring polarity
- ✓ Per-relay Active-LOW polarity setting
- ✓ Per-relay Active-HIGH polarity setting
- ✓ Global Active-LOW override (all channels)
- ✓ Global Active-HIGH override (all channels)
- ✓ Per-relay mode when global mode = 0 (Individual)
- ✓ Immediate re-drive of all pins when global polarity changes
- ✓ Pin-level verification every 30 s (detects glitches / mode loss)
- ✓ Outputs only rewrite on state change (reduces flicker/EMI)
- ✓ Safe boot: all channels forced OFF at startup
- ✓ Custom relay display names (max 15 characters + NUL)
- ✓ Double-click rename in Relays UI with Enter/Escape/blur save
- ✓ XSS-safe name rendering via escapeHtml in browser

B. Manual Control

- ✓ Force channel ON (manual override)
- ✓ Force channel OFF (manual override)
- ✓ Return to Auto (clear override, resume schedules)
- ✓ Manual state persisted in NVS with schedules
- ✓ Manual state checkpointed in criticalState blob
- ✓ Manual overrides restored after reboot if checksum valid
- ✓ Manual changes apply immediately (no 500 ms debounce)
- ✓ Manual mode badge (orange MANUAL) in UI
- ✓ Schedule cache forced inactive while manual is set

C. Scheduling Engine

- ✓ 8 independent schedule slots per relay (Sched 1–8)
- ✓ Start time with hour, minute, second precision
- ✓ Stop time with hour, minute, second precision
- ✓ Per-slot enable/disable checkbox
- ✓ Day-of-week bitmask (Sun–Sat individual toggles)
- ✓ DAY_ALL (0x7F) Everyday shortcut recognition in UI
- ✓ DAY_WEEKDAYS (0x3E) and DAY_WEEKENDS (0x41) constants
- ✓ Day-of-month bitmask (days 1–31 individual toggles)
- ✓ monthDays = 0 means any day of month (filter off)
- ✓ Month-of-year bitmask (Jan–Dec) — v11 feature
- ✓ monthMask = 0 or 0x0FFF means all months
- ✓ Same-day window when start < stop: ON in [start, stop)
- ✓ Overnight window when start > stop: crosses midnight
- ✓ Overnight uses yesterday's DOW bit for post-midnight tail
- ✓ Always-ON when start == stop and today matches days mask
- ✓ OR logic across slots: any matching slot turns relay ON
- ✓ Schedule process interval 250 ms (4 Hz evaluation)
- ✓ Schedule cache rebuild every 1000 ms
- ✓ 500 ms debounce on schedule-driven transitions (anti-chatter)
- ✓ Local-time evaluation (GMT offset + daylight offset applied)
- ✓ Schedules idle safely when clock is invalid (epoch unset)
- ✓ Night / Overnight / Always-ON badges in Relays UI

- ✓ Live badge refresh when toggling days/months/times before Save
- ✓ Save writes one relay's full 8-slot array via REST

D. Timekeeping

- ✓ 64-bit Unix epoch throughout (Year 2106+ / post-2038 safe)
- ✓ `VALID_UNIX_TIME_64` range gate (min ~2001, max uint64)
- ✓ Custom `gmtime64()` civil calendar (Howard Hinnant algorithm)
- ✓ `localtime64()` applying `gmt_offset` + `daylight_offset`
- ✓ `getLocalEpoch()` fast path without struct tm
- ✓ Software internal RTC with microsecond elapsed tracking
- ✓ Drift compensation multiplier (default 1.0)
- ✓ RTC rebase every 5 minutes (`RTC_REBASE_INTERVAL`)
- ✓ Micros wrap handling in rebase and `getCurrentEpoch`
- ✓ Auto-save internal epoch to NVS hourly when offline only
- ✓ DS3231 hardware RTC on I2C SDA=21 / SCL=22
- ✓ `rtc.lostPower()` detection — reject untrusted chip time
- ✓ Year sanity window 2020–2100 for DS3231 reads
- ✓ Boot priority: DS3231 → NVS `last_rtc_epoch` → NONE
- ✓ Push software→DS3231 on NTP/browser sync (`immediateDS3231Sync`)
- ✓ Pull DS3231→software every 1 hour (`DS3231_SYNC_INTERVAL`)
- ✓ NTP 64-bit UDP client (raw sockets, not `configTime`)
- ✓ NTP packet LI/VN/Mode validation; reject LI=3 unsynchronized
- ✓ NTP epoch conversion via 2208988800 offset
- ✓ 4-server fallback pool: Google, Windows, Cloudflare, Facebook
- ✓ Per-server 3 retries; 5 s timeout; 30 s attempt throttle
- ✓ Configurable auto-sync interval 1–24 hours
- ✓ Preferred NTP hostname stored in `sysConfig.ntp_server`
- ✓ Manual Sync NTP button (non-blocking initiate)
- ✓ Browser time sync POST `utc_epoch` from client `Date.now()`
- ✓ Browser sync writes DS3231 and NVS; NTP overrides later
- ✓ Time source tracking: NONE / NTP / BROWSER / RTC
- ✓ UI status dots: green=NTP, blue=browser/RTC, yellow=none
- ✓ Default timezone GMT+8 (28800 seconds)
- ✓ Configurable DST / `daylight_offset` seconds

E. WiFi Station

- ✓ STA enable/disable toggle (persisted in `ExtConfig`)
- ✓ AP-only mode when STA disabled (no internet/NTP)
- ✓ Connect with stored SSID (32) + password (64)
- ✓ Save & Connect from WiFi page
- ✓ Async network scan (`WiFi.scanNetworks true,true`)
- ✓ Scan shows SSID, RSSI, encryption lock icon
- ✓ RSSI bar graphic (4 segments) in UI
- ✓ Click scanned network to fill SSID field
- ✓ Scan pause system: disconnects in-flight STA for 15 s
- ✓ Scan requires STA mode enabled
- ✓ Reconnect state machine with 20 s per-attempt timeout
- ✓ Max 10 reconnect attempts then 5-minute cool-down
- ✓ First-boot attempt skips cool-down penalty
- ✓ Status check every 10 s (`WIFI_CHECK_INTERVAL`)
- ✓ Detect silent disconnect (driver down, flag still up)
- ✓ Live SSID mismatch correction in self-heal
- ✓ WiFi page status: Connected / Not Connected / Station Disabled

F. WiFi Access Point

- ✓ SoftAP always available for local configuration
- ✓ Configurable AP SSID (max 31 chars)
- ✓ Configurable AP password (blank = open; else ≥ 8 chars)

- ✓ WiFi channel selectable 1–13 (default 6)
- ✓ Hidden SSID mode (do not broadcast)
- ✓ AP restart on SSID/password/channel/visibility change
- ✓ liveReconfigureAP if softAPIP becomes 0.0.0.0
- ✓ Forced AP bounce helper restartAPIfNeeded(true)
- ✓ Default SSID: ESP32_16CH_Timer_Switch
- ✓ Default password: 12345678
- ✓ Warning banner in UI before changing AP settings

G. Web User Interface

- ✓ Page: Relays (/) — control + 8 schedule editors
- ✓ Page: WiFi (/wifi) — STA toggle, scan, credentials
- ✓ Page: Time (/ntp) — NTP, offsets, sync buttons
- ✓ Page: AP (/ap) — SoftAP settings
- ✓ Page: GPIO (/gpio) — pins, polarity, global mode
- ✓ Page: System (/system) — diagnostics + resets
- ✓ Shared CSS (/style.css) design system in PROGMEM
- ✓ Sticky gradient header with navigation
- ✓ Live local clock in header (1 Hz poll /api/time)
- ✓ WiFi status dot (green/red) in header
- ✓ Time-source status dot (green/blue/yellow) in header
- ✓ Toast notifications (success green / error red, 3 s)
- ✓ Card grid responsive layout (minmax 340px)
- ✓ Mobile breakpoint @media max-width 500px
- ✓ Info bar tiles on System page (auto-refresh 5 s)
- ✓ All HTML/CSS/JS stored in flash via PROGMEM raw strings
- ✓ Served with send_P to avoid large RAM copies

H. Networking Services

- ✓ HTTP WebServer on port 80
- ✓ Full REST JSON API (GET/POST)
- ✓ Chunked streaming for large /api/relays responses
- ✓ ArduinoJson Static/Dynamic documents sized per endpoint
- ✓ DNSServer captive portal on port 53 wildcard *
- ✓ iOS hotspot-detect.html success page
- ✓ iOS library/test/success.html
- ✓ Android /generate_204 → 302 to AP root
- ✓ Microsoft connecttest.txt + ncsi.txt
- ✓ /success.txt, /canonical.html, /redirect
- ✓ onNotFound → 302 redirect to SoftAP IP /
- ✓ mDNS hostname advertisement .local
- ✓ mDNS TXT: model, version=v11, channels, features
- ✓ Hostname sanitised from AP SSID or custom API set
- ✓ Deferred mDNS restart (2 s) after rename
- ✓ Response cache structure for relays/system/time JSON
- ✓ Connection activity tracking for web health
- ✓ Stale client cleanup every 60 s (up to 5 sockets)

I. Self-Healing & Reliability

- ✓ SelfHealingSystem class with live reconfigure methods
- ✓ smartRecovery tick every 10 s from loop
- ✓ WiFi failure counter → force begin at ≥ 3
- ✓ mDNS re-announce every 5 minutes
- ✓ Full health check every 30 minutes
- ✓ performTargetedRecovery ordered stack repair
- ✓ recoverNTP walks entire server pool
- ✓ recoverRTC re-inits I2C and push/pull time
- ✓ verifyRelayStates digitalRead vs expected level

- ✓ CriticalRelayState NVS checkpoint (magic + checksum)
- ✓ criticalStateDirty coalesced save ($\leq$ every 5 min)
- ✓ Memory cleanup: scanDelete, preferences.end, malloc touch
- ✓ Low-heap emergency cleanup if freeHeap < 20 KB
- ✓ Hourly unconditional memory hygiene
- ✓ Cache invalidation after 5 s stale
- ✓ Service Verify button (/api/reset) without reboot
- ✓ HealthMetrics counters for wifi/ntp/mdns/dns/web

J. Factory Reset & Maintenance

- ✓ Web factory reset POST /api/factory-reset
- ✓ Hardware factory reset: hold BOOT (GPIO0) 5 seconds
- ✓ preferences.clear() wipes entire relay16 namespace
- ✓ initDefaults restores all factory settings
- ✓ Relays forced OFF and manuals cleared on reset
- ✓ AP restarted and targeted recovery after reset
- ✓ No ESP.restart() — device keeps running on defaults
- ✓ Factory reset confirmation dialogs in UI
- ✓ BOOT button uses INPUT_PULLUP active-low edge detect
- ✓ One-shot factoryResetTriggered guard during hold

K. Persistence (NVS)

- ✓ Key sysConfig — SystemConfig packed blob
- ✓ Key relayConfigs — RelayConfig[16] array
- ✓ Key extConfig — ExtConfig packed blob
- ✓ Key gpioConfig — GPIOPinConfig blob
- ✓ Key criticalState — CriticalRelayState blob
- ✓ Magic-number validation on every load
- ✓ Size validation on every getBytes
- ✓ Auto initDefaults if sysConfig corrupt
- ✓ Config version migration v10→v11 (monthMask fill)
- ✓ Field clamps on extConfig (channel, hours, modes)
- ✓ GPIO invalid → reinstall 16 default pins active-low

L. Developer / Internal Features

- ✓ Millis-safe timeHasElapsed / isTimeReached helpers
- ✓ Forward declarations for single-TU Arduino build
- ✓ Packed structs for stable NVS sizeof
- ✓ MAX_RELAYS = 16 compile-time ceiling
- ✓ Inline helpers for pin/polarity/NTP interval
- ✓ Yield points in long handlers and NTP waits
- ✓ scan timeout abort after 10 s
- ✓ NTP async state constants (IDLE/CONNECTING/WAITING)
- ✓ Chip model string exposed as ESP32-38P in API
- ✓ Drift compensation field exposed in /api/system

Total catalogued feature items above: 190 — all implemented in the source sketch.

3. Hardware Requirements & Pinout

3.1 Required Hardware

- **ESP32** development board (38-pin module documented; other ESP32 variants OK if pins match)
- **Relay module** 1–16 channels (opto-isolated recommended for mains)
- **Power supply** adequate for ESP32 + relay coil bank (often external JD-VCC)
- **Optional:** DS3231 RTC module with CR2032 backup battery
- **Optional:** Common ground between ESP32 and relay logic side

3.2 I2C — DS3231 RTC

Item	Value / Description
SDA	GPIO 21
SCL	GPIO 22
VCC	3.3 V
GND	Common ground
Library	RTCLib (RTC_DS3231)
If absent	Firmware continues; uses NTP/browser/software RTC only
lostPower()	If true after battery fail, chip time is not trusted until resync

3.3 BOOT Button (Factory Reset)

Item	Value / Description
Pin	GPIO 0 (BOOT)
Wiring	Active LOW to ground when pressed; INPUT_PULLUP in firmware
Hold time	5,000 ms continuous
Action	Full NVS wipe + factory defaults + live service recovery

3.4 Default Relay GPIO Map

Logical relay index maps to these GPIOs out of the box. Change anytime on the GPIO page or by editing DEFAULT_RELAY_PINS in source.

Relay	GPIO	Label	Relay	GPIO	Label
1	32	IN1	9	1	IN9
2	33	IN2	10	3	IN10
3	25	IN3	11	19	IN11
4	26	IN4	12	18	IN12
5	27	IN5	13	5	IN13
6	14	IN6	14	4	IN14
7	13	IN7	15	2	IN15
8	23	IN8	16	15	IN16

Warning: GPIO 1 (TX0) and GPIO 3 (RX0) are UART0. Using them as relays disables USB-serial logging after pinMode(OUTPUT). GPIO 2 is often the onboard LED on DevKit boards.

Note: Most multi-channel relay boards are **Active LOW** (energise when IN is 0 V). That matches the firmware default activeLow=true and Global Mode 0/1.

3.5 Polarity Model

```

logical ON → relay coil energised (load path closed for NO contacts)
setRelayOutput(i, state):
  level = isActiveLow(i) ? !state : state
  digitalWrite(pin, level)

isActiveLow(i):
  global_active_mode==1 → always true (all LOW)
  global_active_mode==2 → always false (all HIGH)
  else → gpioConfig.activeLow[i]
```


4. Getting Started / First Boot

4.1 Flash the Firmware

- Open the .ino in Arduino IDE or PlatformIO with ESP32 board support
- Install libraries: ArduinoJson, RTCLib (Adafruit), (WiFi/WebServer/DNS/Preferences/ESPmDNS are core)
- Select your ESP32 board and correct COM port; upload
- Serial monitor optional — note GPIO1/3 usage may mute logs after setup pinMode

4.2 First Connection

- On boot, SoftAP starts: **SSID ESP32_16CH_Timer_Switch** / password **12345678**
- Connect your phone/PC to that WiFi network
- Open browser to **http://192.168.4.1/** (typical SoftAP IP) or wait for captive portal popup
- Alternatively try **http://<mdns-host>.local/** if mDNS works on your OS
- You land on the **Relays** page

4.3 Recommended First-Time Setup Order

Step	Page	Action
1	GPIO	Confirm pin map matches your wiring; set polarity / global mode
2	AP	Change AP SSID/password if deploying beyond lab (clients will reconnect)
3	WiFi	Enable Station, scan, save home/office SSID for internet/NTP
4	Time	Set GMT offset; Sync NTP (or Sync Browser if offline)
5	System	Verify STA IP, time source, DS3231 present, heap healthy
6	Relays	Name channels; program schedules; test Manual ON/OFF/Auto

4.4 Boot Sequence (What Firmware Does)

#	Action	Why
1	initRTC() — probe DS3231	Best time source if battery-backed
2	loadGPIOConfig()	Restore pin map / polarity
3	BOOT pin INPUT_PULLUP	Enable hardware factory reset
4	All relay pins OUTPUT + OFF	Safe de-energised start
5	initScheduleDefaults ×16	RAM baseline before NVS overlay
6	loadConfiguration + loadExtConfig	SSID, schedules, AP, flags
7	Time: DS3231 else NVS epoch else NONE	Clock hierarchy
8	WiFi mode AP_STA or AP	Per sta_enabled
9	WiFi.begin if STA credentials exist	Background join
10	softAP with channel/hidden	Always-on config access
11	startMDNS + DNSServer + WebServer	Discovery + captive + UI
12	updateScheduleCache + restoreCriticalState	Schedules + manual restore

5. Web User Interface — All 6 Pages

The UI is a multi-page SPA-like set of static HTML documents with vanilla JS. Shared look-and-feel comes from /style.css. Every page header shows navigation, WiFi dot, time-source dot, and a live clock.

5.1 Common Chrome (All Pages)

- **Nav links:** Relays · WiFi · Time · AP · GPIO · System
- **Clock:** fetches GET /api/time every 1 second
- **WiFi dot:** green if d.wifi true, else red
- **Time dot:** green=ntp, blue=browser or rtc, yellow=none/other
- **Toast:** #toast element, classes show ok|er, auto-hide 3 s
- **Theme:** blue gradient header, white cards, Material-like badges/buttons

5.2 Relays Page — /

Purpose: Primary operations console for all channels and schedules.

Layout

- Responsive card grid — one card per configured relay
- Empty state if count=0 with link to GPIO page
- Each card: name (double-click edit), status badge, ON/OFF/Auto buttons
- Scrollable list of 8 schedule editors (Sched 1–8)
- Save button per card persists that relay only

Status Badges

Badge	Condition	Color
ON	Not manual and state true	Green
OFF	Not manual and state false	Red
MANUAL	manualOverride true	Orange

Schedule Editor Fields (×8)

- Enable checkbox
- Start time input type=time step=1 (HH:MM:SS)
- Stop time input type=time step=1
- Days of Week — 7 toggle chips Sun...Sat
- Days of Month — 31 toggle chips 1...31
- Months of Year — 12 toggle chips Jan...Dec
- Dynamic night badge: Always ON / Overnight / day summary

Client JS Functions (Relays)

Function	Role
toast / tick	Notifications + 1 Hz status
escapeHtml	Safe name display
load / render	GET /api/relays and build DOM
toTS / fromTS	Time string ↔ h,m,s
dayMaskToStr / monthDayMaskToStr / monthMaskToStr	Human summaries
nightBadge	Always/Overnight info HTML
startEditName / cancelEdit / saveNameEdit	Inline rename → POST name
toggleDay / toggleMonthDay / toggleMonth	XOR bits in local model
uf	Update enable/start/stop fields
mc	POST manual ON/OFF
ra	POST reset to Auto
save	POST full schedules array for one relay

5.3 WiFi Page — /wifi

Purpose: Station mode control, scanning, and credentials.

- Large STA ON/OFF toggle button (blue/red gradients)
- When OFF: station config dimmed/disabled; alert explains AP-only mode
- SSID text field + Scan button
- Live network list sorted by RSSI with bars + lock icon
- Password field + Save & Connect
- Status alert: Connected (IP+RSSI) / Not Connected / Station Disabled
- Toggle path POSTs {sta_enabled}; credential path POSTs {ssid,password}

Note: Scanning is blocked until Station Mode is ON. During connect, scan may pause STA for 15 s.

5.4 Time Page — /ntp

Purpose: Configure time authority and timezone.

- NTP Server hostname field (fallback pool still used by engine)
- GMT Offset seconds (e.g. 28800 = UTC+8)
- Daylight Saving seconds
- Auto Sync hours 1–24
- Buttons: Save Settings · Sync NTP · Sync Browser
- Status panel shows source accuracy + DS3231 detection on GPIO21/22
- Browser sync sends `Math.floor(Date.now()/1000)` as `utc_epoch`

5.5 AP Page — /ap

Purpose: SoftAP identity and RF parameters.

- AP SSID, AP Password, Channel select 1–13, Visibility Visible/Hidden
- Warning: changes restart AP and disconnect clients
- Password validation: empty allowed (open); if set must be ≥ 8 characters
- On success with `restarted=true`: toast + reload after 4 s

5.6 GPIO Page — /gpio

Purpose: Channel count, pin map, polarity.

- Global Active Level Mode select: Per-Relay / All LOW / All HIGH
- Add Relay: dropdown of available GPIOs + Add button
- List: Relay N · GPIO · Active LOW/HIGH · Toggle · Remove
- Global mode shows amber banner and disables per-relay toggle buttons
- Reset to Default restores 16-pin `DEFAULT_PINS` array
- Delete confirms that schedules shift/renumber
- Max 16 relays hard limit messaging

5.7 System Page — /system

Purpose: Live diagnostics and destructive maintenance.

Info tiles (refresh every 5 s)

- ✓ STA IP
- ✓ AP IP
- ✓ Free Heap (KB)
- ✓ Uptime
- ✓ WiFi RSSI (+ quality text)
- ✓ Time Source (colour-coded)
- ✓ UTC Epoch
- ✓ NTP Last Sync age
- ✓ Last Browser Sync age
- ✓ NTP Server
- ✓ Chip Model
- ✓ mDNS Hostname
- ✓ Relay Count
- ✓ GMT Offset
- ✓ Drift Comp
- ✓ GPIO Mode

- ✓ DS3231 RTC present
- ✓ DS3231 Last Sync
- ✓ Sync Direction
- ✓ WiFi Station status

Actions

- **Verify Services** — POST /api/reset — live recovery, no reboot, no wipe
- **Factory Reset** — POST /api/factory-reset — wipe all settings, stay running
- Help text mentions BOOT button 5 s hardware factory reset

6. Relay Control Features

6.1 Logical State Model

The firmware always reasons in **logical** ON/OFF. Polarity conversion happens only at the last millimetre inside `setRelayOutput()`. UI badges and API state fields are logical.

```
if (manualOverride) state = manualState;
else state = scheduleEngineShouldBeOn;
drive pin with polarity applied
```

6.2 Manual Override Behaviour

Action	API	Effect
ON	POST /api/relay/manual {relay, state:true}	override=true, state=true, pin ON, save NVS, dirty critical
OFF	POST /api/relay/manual {relay, state:false}	override=true, state=false, pin OFF, save NVS, dirty critical
Auto	POST /api/relay/reset {relay}	override=false, recompute schedules, save NVS

While override is true, `processRelaySchedules` ignores time windows for that channel and drives `manualState` immediately (no debounce). `scheduleActiveCache[i]` is forced false on manual set so cache consumers do not fight the override.

6.3 Naming

- Stored in `relayConfigs[i].name[16]` (15 usable chars)
- Default "Relay 1" ... "Relay 16" from `initScheduleDefaults`
- UI: double-click title → input maxLength 15 → POST /api/relay/name
- Empty name coerced back to "Relay N"

6.4 Output Path Details

- `lastRelayOutputs[]` remembers last logical drive — skips redundant `digitalWrite`
- `relayOutputsInitialized` gates first-time drive
- `updateRelayOutputs()` bulk helper exists; live path is `processRelaySchedules()`
- `verifyRelayStates()` reads GPIO and corrects mismatches every 30 s

7. Scheduling Engine (Full Rules)

The schedule engine is the heart of the product. It evaluates local civil time against up to 8 slots per channel and produces a boolean `shouldBeOn`.

7.1 Data per Slot

Item	Value / Description
startHour/Minute/Second	Start time components (0–23 / 0–59 / 0–59)
stopHour/Minute/Second	Stop time components
enabled	false → slot ignored
days (uint8)	Bit0=Sun ... Bit6=Sat
monthDays (uint32)	Bit0=day1 ... Bit30=day31; 0=any day
monthMask (uint16)	Bit0=Jan ... Bit11=Dec; 0 or 0x0FFF=all months

7.2 Evaluation Order (per slot)

A slot contributes ON only if **all** of the following pass:

- `enabled == true`
- Month filter: if `monthMask` ≠ 0, bit for current month must be set
- DOM filter: if `monthDays` ≠ 0, bit for (`mday`–1) must be set
- Time-window rule (below) must match
- Across slots: result is OR — first success sets `hasActive/shouldBeOn` and breaks

7.3 Time-Window Rules

Condition	Rule	Example
start == stop	Always ON for the whole day if DOW matches	08:00:00–08:00:00 on Mon → ON all Monday
start < stop	ON when start ≤ now < stop and DOW matches today	09:00–17:00 office hours
start > stop	Overnight: ON if (now ≥ start & today) OR (now < stop & yesterday DOW)	22:00–06:00 night lights

Note: Same-day windows use half-open interval [start, stop): at exactly stop the channel turns OFF.

7.4 Day-of-Week Constants

Constant	Value	Meaning
DAY_SUNDAY ... DAY_SATURDAY	1<<0 ... 1<<6	Individual days
DAY_ALL	0x7F	Everyday
DAY_WEEKDAYS	0x3E	Mon–Fri
DAY_WEEKENDS	0x41	Sat+Sun

7.5 Month Constants (v11)

MONTH_JANUARY (1<<0) ... MONTH_DECEMBER (1<<11), MONTH_ALL = 0x0FFF. Older configs migrating from version < 10 get monthMask filled with MONTH_ALL automatically.

7.6 Engine Timing

Item	Value / Description
processRelaySchedules()	Every 250 ms from loop
updateScheduleCache()	Every 1000 ms (also on time sync / save)
Debounce	500 ms between schedule-driven transitions
Manual path	Immediate, no debounce
Timezone basis	getCurrentEpoch() + gmt_offset + daylight_offset
Invalid clock	epoch < MIN_UNIX_TIME_64 → engine returns without driving schedules

7.7 Practical Schedule Examples

Goal	Configuration
Office lights Mon–Fri 8–18	enabled; start 08:00:00 stop 18:00:00; days=weekdays; months=all; DOM=any
Pump every day 06:00–06:15	start 06:00:00 stop 06:15:00; days=all
Night sign 20:00–06:00	start 20:00:00 stop 06:00:00 (overnight); days as needed
Always on in December	start==stop; monthMask=Dec only; days=all
1st of month only 12:00–13:00	monthDays bit0 set; start 12:00 stop 13:00; days=all
Two windows same day	Use Sched1 and Sched2 both enabled — OR logic

8. GPIO & Polarity Management

8.1 Capabilities

- View all configured pins and active levels
- Add a pin from the allowed list (must be unique, count < 16)
- Delete a pin by index (array compaction — later relays renumber)
- Toggle per-relay active level (when global mode = 0)
- Set global mode 0/1/2 with immediate re-drive
- Reset to factory 16-pin map (preserves schedules by index where possible)

8.2 Allowed / Default Pins

The API availablePins list and the UI DEFAULT_PINS constant match: 32, 33, 25, 26, 27, 14, 13, 23, 1, 3, 19, 18, 5, 4, 2, 15. The UI hides pins already in use from the Add dropdown.

8.3 Global Active Mode

Mode	Name	Behaviour
0	Per-Relay (Individual)	Use gpioConfig.activeLow[i] per channel
1	Global Active LOW	All channels treated active-low; UI disables toggles
2	Global Active HIGH	All channels treated active-high; UI disables toggles

8.4 Side Effects on GPIO Changes

- Add/Delete/Save/Toggle: pinMode OUTPUT, force logical OFF on affected channels
- saveGPIOConfig + saveConfiguration + updateScheduleCache
- criticalStateDirty = true; responseCache invalidated
- Delete shifts relayConfigs with pins — schedules move with the slot

9. WiFi Station & Access Point

9.1 Dual-Mode Model

sta_enabled	WiFi.mode	Internet / NTP	How to open UI
1 (default)	WIFI_AP_STA	Yes when joined	STA IP and/or AP IP + mDNS
0	WIFI_AP	No	AP IP + captive portal + mDNS

9.2 Station Connection State Machine

- **beginWiFiConnect:** guards (enabled, SSID, not cooling down, not connecting, not paused) → disconnect → begin
- **Connecting:** poll status; keep handleClient/yield so UI stays alive on AP
- **Success:** flags clear; attempts=0; tryNTPSync immediately
- **Timeout 20 s:** stop attempt; if attempts≥10 and not firstAttempt → cool-down 5 minutes
- **Every 10 s:** detect disconnects and schedule reconnect when allowed
- **Scan pause:** if scan requested while connecting, disconnect 15 s; auto-clear pause

9.3 Access Point Features

- Always started in setup with configured SSID/password/channel/hidden
- Channel clamped to 1–13 (default 6)
- Blank password → open network (WiFi.softAP(ssid, NULL, ...))
- Any AP identity change → forced softAPdisconnect + recreate + DNS/mDNS refresh
- Self-heal recreates AP if IP is 0.0.0.0

9.4 Scanning

- POST /api/wifi/scan starts async scan (202 scanning)
- GET /api/wifi/scan polls; returns networks[{ssid,rssi,enc}] up to 30
- Hidden networks included (scanNetworks second arg true)

- Empty SSID results skipped; scanDelete after read
- 10 s timeout aborts stuck scans in loop

10. Timekeeping System

10.1 Four Time Sources

Source	Code	Accuracy	Needs	Persists across power?
None	0	—	—	No clock
NTP	1	High	STA + Internet	Via DS3231/NVS after sync
Browser	2	Moderate	Any client with clock	Via DS3231/NVS after sync
RTC	3	High (DS3231) / fair (software)	DS3231 or prior epoch	DS3231 battery or NVS epoch

10.2 Authority Priority

BOOT:

- 1) DS3231 present, not lostPower, year 2020..2100 → TIME_SOURCE_RTC
- 2) else NVS last_rtc_epoch valid → software RTC
- 3) else TIME_SOURCE_NONE (schedules wait)

RUNTIME:

- Browser sync → TIME_SOURCE_BROWSER (NTP can override later)
 NTP success → TIME_SOURCE_NTP (preferred when online)
 Hourly DS3231 pull → may set RTC if not currently NTP

10.3 Software Internal RTC

- Base: internalEpoch (uint64 UTC seconds)
- Anchor: rtcMicrosAtLastSync + driftCompensation
- getCurrentEpoch() extrapolates elapsed micros × drift, rounds ≥0.5 s up
- performRTCReabase() folds elapsed into base every 5 minutes
- autoSaveInternalRTC() writes NVS hourly only if no NTP and no valid DS3231
- saveRTCState() on successful external sync

10.4 NTP Client Features

- Raw UDP port 123; local bind 2390
- 48-byte packet; Mode 3 client request; validates Mode 4 response
- Rejects leap indicator 3 (clock not synchronized)
- Rejects epochs before 2020-01-01 (1577836800)
- Pool rotation on success and full failover on failure
- tryNTPSync throttled to 30 s; interval between successes = getNTPInterval()
- handleSyncNTP resets timers so loop picks up quickly (non-blocking)
- recoverNTP used by self-healing walks all servers

10.5 Browser Sync Features

- Endpoint POST /api/time/browser-sync
- Body: {"utc_epoch": <number>}
- Validates range; sets BROWSER; syncInternalRTC; forces BROWSER again
- Clears lastNTPSync so NTP can supersede when available
- Updates DS3231; returns local_time string for instant UI feedback

10.6 Timezone

- sysConfig.gmt_offset — seconds east of UTC (default 28800 = Philippines/Singapore UTC+8)
- sysConfig.daylight_offset — extra seconds for DST
- Schedules and UI clock always use local time = UTC + both offsets

10.7 64-bit / Year 2106+

Standard 32-bit Unix time overflows in 2038. This firmware stores and computes epochs as uint64_t, implements gmtime64 without relying on 32-bit time_t, and only narrows to 32-bit when writing DS3231 DateTime (valid through 2106).

11. mDNS Discovery

- Started in setup via startMDNS()
- Default hostname derived from AP SSID: lowercased, spaces/_ → '-', strip invalid, max 31
- Fallback hostname "esp32"
- Service: _http._tcp:80
- TXT records: model=ESP32, version=v11, channels=<count>, features=monthmask,64bit
- API GET/POST /api/mdns and POST /api/mdns/restart
- Rename schedules restart after 2 s delay so HTTP response completes
- Self-heal re-adds service periodically

Note: mDNS (.local) support varies: works well on many desktops; some Android versions are limited.

12. Captive Portal

When a client joins the SoftAP, the DNSServer answers all DNS queries with the AP IP. OS captive-detection URLs are implemented so phones open a browser sheet automatically.

Path	Client OS	Response
DNS * → AP IP	All	DNSServer port 53
/hotspot-detect.html	iOS/macOS	Success HTML
/library/test/success.html	iOS	Success HTML
/generate_204	Android	302 → http://AP/
/success.txt	Various	success
/canonical.html	Various	302 → AP/
/connecttest.txt	Windows	Microsoft Connect Test
/ncsi.txt	Windows	Microsoft NCSI
/redirect	Various	302 → AP/
any unknown	All	onNotFound 302 → AP/

Warning: There is no login page authentication. Anyone on the AP (or STA LAN) can control relays. Use a strong AP password and/or place the device on a trusted network segment.

13. Self-Healing & Reliability

13.1 Philosophy

Long-running field devices lose WiFi, drop mDNS, glitch GPIO, or fragment heap. Rather than reboot on every fault, the firmware runs a SelfHealingSystem that repairs subsystems in place while schedules continue.

13.2 smartRecovery() — Every 10 Seconds

- If STA should be up but is not (and not connecting/paused): ++wifiFailures; at ≥ 3 force WiFi.begin
- If driver connected: sync wifiConnected flag; clear failures
- Every 5 minutes: re-add mDNS HTTP service
- Every 30 minutes: mDNS announce + flush dirty criticalState
- Always: verifyRelayStates + liveReconfigureAP

13.3 performTargetedRecovery() — On Demand

Used by System → Verify Services and after factory reset:

- liveReconfigure WebServer → DNS → mDNS → WiFi → AP (50 ms gaps)
- recoverRTC
- verifyRelayStates
- Zero health failure counters

13.4 Critical Relay State Checkpoint

- Structure holds last logical outputs + manual override flags + timestamp + checksum
- Magic 0xDEADBEEF; checksum mixes state bits XOR timestamp
- Saved when criticalStateDirty and ≥ 5 minutes since last save (also on 30 min health)
- On boot restore: only channels with manualOverrides[i] are re-applied
- Schedule-driven channels recompute from clock instead

13.5 Memory Management Features

- cleanupStaleResources every 30 s: orphan scan abort, cache clear, deep clean
- performMemoryCleanup: scanDelete, yield bursts, preferences.end, malloc/free touch
- checkAndCleanMemory: track minFreeHeap; emergency if < 20 KB; hourly cleanup
- Web idle 5 minutes → liveReconfigureWebServer touch

13.6 Relay Pin Verification

Every 30 s, for each channel: compute expected electrical level from logical state + polarity; digitalRead pin; if mismatch, setRelayOutput again. Protects against EMI, accidental pinMode loss, or external drive conflicts.

14. Factory Reset (Web + Hardware)

14.1 What Gets Wiped

- Entire NVS namespace "relay16" via preferences.clear()
- Then initDefaults() writes factory sysConfig, gpioConfig, empty schedules, names
- extConfig reloaded (defaults: ch6, sync 1 h, visible AP, per-relay polarity, STA on)

14.2 What Happens After Wipe

- All relays OFF; manual flags cleared
- If default STA enabled and SSID empty — stays on AP until you configure WiFi
- AP restarted with default SSID/password
- performTargetedRecovery + updateScheduleCache
- **Device does not reboot** — continues running

14.3 Triggers

Trigger	How	UI feedback
Web	System page → Factory Reset → confirm → POST /api/factory-reset	Toast + redirect home ~7 s
Hardware	Hold BOOT (GPIO0) 5 seconds	No web toast; settings simply reset; reconnect to default AP if needed

14.4 Non-Destructive Alternative

Verify Services runs `performTargetedRecovery` only — repairs networking/RTC/pins without erasing configuration.

15. System Information & Diagnostics

GET /api/system powers the System page tiles. Fields returned:

JSON field	Meaning
ip	STA IPv4 string
ap_ip	SoftAP IPv4 string
uptime	seconds since boot (millis/1000)
freeHeap	ESP.getFreeHeap() bytes
utcEpoch	stringified uint64 current UTC epoch
timeSource	None NTP Browser RTC
ntpServer	configured preferred server
ntpSyncAge	seconds since last NTP; 0xFFFFFFFF = never
browserSyncAge	seconds since browser sync; never sentinel
rtcSyncAge	seconds since DS3231 touch; never sentinel
wifiConnected	bool
wifiSSID	configured STA SSID
rssi	dBm if connected else 0
version	EEPROM_VERSION (11)
chipModel	ESP32-38P
mdnsHostname / mdnsStarted	discovery state
relayCount / maxRelays	active vs ceiling 16
gmtOffset	seconds
driftComp	float drift compensation
globalActiveMode	0/1/2
rtcPresent	DS3231 detected
staEnabled	station mode flag

16. Complete REST API Reference

Base URL: `http://<device-ip>` or `http://<host>.local` — all JSON unless noted. POST bodies use Content-Type: `application/json` and are read from `server.arg("plain")`.

16.1 Pages & Static Assets

Method	Path	Response
GET	/	Relays HTML
GET	/wifi	WiFi HTML
GET	/ntp	Time HTML
GET	/ap	AP HTML
GET	/gpio	GPIO HTML
GET	/system	System HTML
GET	/style.css	Shared CSS

16.2 Relay API

GET /api/relays

List all active relays with state, manual flag, name, pin, and 8 schedules. Uses chunked transfer encoding for large payloads.

Response: [{ state, manual, name, pin, schedules:[{startHour,startMinute,startSecond,stopHour,stopMinute,stopSecond,enabled,days,monthDays,monthMask}×8] }]

POST /api/relay/manual

Force manual ON/OFF.

Request: { "relay":0, "state":true }

Response: { "success":true }

Errors: 400 No data / Bad JSON / Invalid relay

POST /api/relay/reset

Clear manual override; return to schedule control.

Request: {"relay":0}

Response: {"success":true}

POST /api/relay/save

Replace all schedule slots for one relay (up to 8). monthMask defaults to MONTH_ALL if omitted.

Request: {"relay":0,"schedules":[...]}

Response: {"success":true}

Errors: 400; uses StaticJsonDocument<5120>

POST /api/relay/name

Set display name (max 15 chars stored).

Request: {"relay":0,"name":"Pump A"}

Response: {"success":true}

16.3 Time API

GET /api/time

Compact status for 1 Hz UI tick.

Response: {"time":"HH:MM:SS","wifi":bool,"ntp":bool,"timeSource":"none|ntp|browser|rtc","rtcPresent":bool,"rtcSynced":bool,"rtcSyncAge":sec}

POST /api/time/browser-sync

Set clock from client UTC epoch; updates DS3231; returns local_time.

Request: {"utc_epoch":1730000000}

Response: {success, utc_epoch, local_time, gmt_offset, time_source, rtc_present, rtc_synced, drift}

Errors: 400 invalid/missing epoch

16.4 WiFi API

GET /api/wifi

Station status.

Response: {"ssid","connected","ip","rssi","sta_enabled"}

POST /api/wifi (toggle)

Enable/disable station mode live.

Request: {"sta_enabled":true}

Response: {"success":true,"sta_enabled":true}

POST /api/wifi (credentials)

Save SSID/password; reconnect if STA enabled and values changed. Empty password clears PSK.

Request: {"ssid":"MyNet","password":"secret"}

Response: {"success":true}

Errors: 400 Invalid SSID

POST /api/wifi/scan

Start async scan. Pauses in-flight STA connect if needed.

Response: 202 {"scanning":true}

Errors: 400 WiFi station is disabled

GET /api/wifi/scan

Poll scan. When done returns up to 30 networks.

Response: {"scanning":false,"networks":[{"ssid","rssi","enc"]}}

16.5 NTP API

GET /api/ntp

Read NTP-related settings (also echoes globalActiveMode).

Response: {"ntpServer","gmtOffset","daylightOffset","syncHours","globalActiveMode"}

POST /api/ntp

Save server + offsets + optional syncHours 1–24.

Request: {"ntpServer":"time.google.com","gmtOffset":28800,"daylightOffset":0,"syncHours":1}

POST /api/ntp/sync

Initiate NTP sync (non-blocking). loop performs actual UDP exchange.

Response: {"success":true,"message":"NTP sync initiated"}

Errors: 400 WiFi not connected or station disabled

16.6 AP API

GET /api/ap

Read AP settings (includes password — local trust model).

Response: {"ap_ssid","ap_password","ap_channel","ap_hidden"}

POST /api/ap

Update SSID/password/channel/hidden; restart AP if anything changed.

Request: {"ap_ssid":"...", "ap_password":"...", "ap_channel":6, "ap_hidden":false}

Response: {"success":true, "restarted":true|false}

16.7 GPIO API

GET /api/gpio

Full GPIO configuration.

Response: {"count","maxRelays","pins":[],"activeLow":[],"availablePins":[]}

POST /api/gpio/save

Replace pin list (e.g. reset defaults). Preserves meta by index when possible.

Request: {"pins":[32,33,...]}

POST /api/gpio/add

Append a new relay pin.

Request: {"pin":27}

Errors: 400 max reached / pin in use

POST /api/gpio/delete

Remove relay by index; compact arrays.

Request: {"index":3}

POST /api/gpio/toggle-active-low

Flip per-relay polarity; forces channel OFF.

Request: {"index":0}

Response: {"success":true, "activeLow":true|false}

GET /api/gpio/global-mode

Read global polarity mode.

Response: {"mode":0}

POST /api/gpio/global-mode

Set mode 0/1/2; re-drive all pins with same logical states.

Request: {"mode":1}

16.8 mDNS API

GET /api/mdns

Hostname status.

Response: {"hostname","started","url"}

POST /api/mdns

Set hostname (sanitised).

Request: {"hostname":"relay-panel"}

POST /api/mdns/restart

Force mDNS reconfigure.

Response: {"success":true}

16.9 System API

GET /api/system

Full diagnostics object (see §15).

POST /api/reset

Live service verification (targeted recovery), no wipe.

Response: {"success":true, "message":"Performing live service verification..."}

POST /api/factory-reset

Wipe NVS, restore defaults, recover services, no reboot.

Response: {"success":true, "message":"Factory reset complete..."}

16.10 Captive Portal Routes

See §12 table — all GET, return success HTML, plain text, or 302 redirects.

17. Configuration & NVS Persistence

17.1 Namespace & Keys

Key	Content	Written by
sysConfig	SystemConfig (magic, version, STA/AP creds, NTP, offsets, epoch, drift, hostname)	saveConfiguration, saveRTCState, initDefaults
relayConfigs	RelayConfig[16] schedules + manual + names	saveConfiguration
extConfig	channel, ntp hours, hidden, global mode, sta_enabled	saveExtConfig
gpioConfig	pins[], count, magic, activeLow[]	saveGPIOConfig
criticalState	manual snapshot + checksum	saveCriticalState

17.2 SystemConfig Fields

Item	Value / Description
magic	0x1234
version	11
sta_ssid[32] / sta_password[64]	Station credentials
ap_ssid[32] / ap_password[32]	SoftAP credentials
ntp_server[48]	Preferred NTP host
gmt_offset / daylight_offset	int32 seconds
last_rtc_epoch	uint64 software clock persistence
rtc_drift	float
hostname[32]	hostname seed

17.3 ExtConfig Fields

Item	Value / Description
magic	0xEC
ap_channel	1–13
ntp_sync_hours	1–24
ap_hidden	0/1
global_active_mode	0/1/2
sta_enabled	0/1
reserved[26]	future expansion

17.4 Load Validation & Migration

- Bad sysConfig magic/size → initDefaults()
- version < 11: if < 10 fill all monthMask=MONTH_ALL; set version 11; save
- Bad relayConfigs size → re-init all schedule defaults
- Bad extConfig → defaults channel 6, sync 1 h, visible, mode 0, STA on
- Bad gpioConfig → 16 default pins, all activeLow true
- Bad criticalState checksum/magic → skip restore (safe)

18. Default Values Reference

Setting	Factory default
AP SSID	ESP32_16CH_Timer_Switch
AP Password	12345678
AP Channel	6
AP Hidden	No (0)
STA Enabled	Yes (1)
STA SSID / Password	(empty)

Setting	Factory default
NTP Server	time.google.com
NTP fallback pool	google, windows, cloudflare, facebook
GMT Offset	28800 (UTC+8)
Daylight Offset	0
NTP Sync Interval	1 hour
Hostname seed	esp32
Relay count	16
Default GPIOs	32,33,25,26,27,14,13,23,1,3,19,18,5,4,2,15
activeLow (each)	true
global_active_mode	0 (per-relay)
Schedule enabled	all false
Schedule days	DAY_ALL (0x7F)
Schedule monthDays	0 (any)
Schedule monthMask	MONTH_ALL (0x0FFF)
Schedule times	00:00:00 (zeroed)
Manual override	false
Relay names	Relay 1 ... Relay 16
Drift compensation	1.0
Time source	NONE until RTC/NTP/browser

19. Runtime Architecture

19.1 Main Loop Phases

Phase	When	Work
Input	every pass	checkBootButton
Network I/O	every pass	handleClient + DNS process
Activity stamp	on client	lastConnectionActivity
Socket cleanup	60 s	stop leftover clients; scanDelete
Memory	30 s	cleanupStaleResources
Web health	60 s	checkWebServerHealth
Self-heal	10 s	smartRecovery
Heap monitor	5 min / 1 h	checkAndCleanMemory
DS3231 pull	1 h	internalEpoch ← chip if valid
RTC autosave	1 h offline	autoSaveInternalRTC
mDNS rename	deadline	restartMDNS
Scan watchdog	10 s	abort stuck scan
STA FSM	while connecting	success→NTP; timeout→backoff
Pause end	deadline	clear wifiPausedForScan
WiFi check	10 s	reconnect / detect
NTP	interval/retry	tryNTPSync
Schedules	250 ms	processRelaySchedules
Critical NVS	5 min if dirty	saveCriticalState
Yield	end	WiFi stack time

19.2 Relay Decision Data Flow

```

Clock (NTP/Browser/DS3231/Software)
↓ getEpoch + timezone
Schedule rules (8 slots × filters × windows)
↓ OR → shouldBeOn
Manual override? → manualState : shouldBeOn
↓ debounce if schedule-driven
setRelayOutput → polarity → digitalWrite(GPIO)

```

19.3 Module Map

Module	Responsibilities
Time core	gmtime64, getEpoch, syncInternalRTC, NTP, DS3231
Schedule core	updateScheduleCache, processRelaySchedules
Output core	setRelayOutput, isActiveLow, verifyRelayStates
WiFi core	STA FSM, scan pause, AP lifecycle
Web core	setupWebServer, all handlers, PROGMEM pages
Heal core	SelfHealingSystem methods, criticalState
Persist core	Preferences load/save, initDefaults, migration

20. Timing & Interval Reference

Constant / Behaviour	Interval	Purpose
SCHEDULE_PROCESS_INTERVAL	250 ms	Evaluate schedules / drive pins
SCHEDULE_CACHE_INTERVAL	1 s	Rebuild calendar cache
Relay schedule debounce	500 ms	Anti-chatter
WIFI_CHECK_INTERVAL	10 s	STA health
WIFI_CONNECT_TIMEOUT	20 s	Single join attempt

Constant / Behaviour	Interval	Purpose
Reconnect cool-down	5 min	After 10 failures
WIFI_PAUSE_DURATION	15 s	Scan pause
NTP_RETRY_INTERVAL	30 s	Min gap between attempts
NTP_TIMEOUT / server timeout	5 s	UDP wait
NTP_RETRY_COUNT	3	Per server inside getNTP64Time
getNTPInterval()	1–24 h	Success cadence
RTC_REBASE_INTERVAL	5 min	Software clock fold
DS3231_SYNC_INTERVAL	1 h	Pull from chip
INTERNAL_RTC_SAVE_INTERVAL	1 h	Offline epoch NVS save
MEMORY_CLEANUP_INTERVAL	30 s	Stale resource cleanup
Heap check	5 min	minFreeHeap + low-heap clean
Hourly memory clean	1 h	Unconditional hygiene
smartRecovery	10 s	Self-heal tick
verifyRelayStates	30 s	Pin audit
mDNS re-announce	5 min	Keep discovery alive
Full health check	30 min	Deep maintenance
Critical save throttle	5 min	NVS wear control
Web idle reconfigure	5 min	No client activity
Client socket cleanup	60 s	Drop leftovers
MDNS_RESTART_DELAY	2 s	After hostname change
Scan watchdog	10 s	Abort stuck scan
FACTORY_RESET_HOLD	5 s	BOOT button
UI clock poll	1 s	tick()
UI system poll	5 s	loadSys()
UI scan poll	2.5 s	pollScan()

21. Security & Trust Model

- **No HTTP authentication** on the web UI or API
- **No TLS/HTTPS** — all traffic plaintext on local network
- Trust boundary = WiFi association (AP password and/or upstream LAN)
- AP password returned by GET /api/ap (anyone on network can read it)
- STA password not returned by GET /api/wifi (only ssid/status)
- Factory reset is intentionally easy (web + BOOT) — physical access = full control
- XSS mitigation: relay names escapeHtml on Relays page
- JSON input validated for types/ranges on sensitive fields
- Recommend: strong AP password, isolate IoT VLAN, disable STA if unused

Warning: Do not expose the device's port 80 to the public internet. It is a local control surface, not a hardened cloud endpoint.

22. Edge Cases & Error Handling

Situation	Firmware behaviour
millis() wrap ~49.7 days	Unsigned subtraction in timeHasElapsed / isTimeReached
micros() wrap ~71.6 min	Detected in RTC rebase/getCurrentEpoch; recompute elapsed
Corrupt NVS blob	Magic/size fail → defaults or field clamps; no crash
DS3231 missing	rtcPresent=false; NTP/browser/software only
DS3231 lostPower	Time not trusted until external sync adjust()
Clock never set	Schedules do not run; manual control still works
All NTP servers fail	Counters++; retry every 30 s while online
WiFi 10 failures	5-minute give-up window
Scan while connecting	pauseWiFiForScan 15 s
freeHeap < 20 KB	Emergency performMemoryCleanup
GPIO level mismatch	verifyRelayStates rewrites expected level
AP IP 0.0.0.0	liveReconfigureAP recreates SoftAP
Overnight schedule	Yesterday DOW bit for post-midnight segment
monthDays = 0	Any day of month
monthMask = 0	Treated as all months (filter only if non-zero)
start == stop	Always ON that day
Delete middle relay	Higher indices shift down with their schedules
Global polarity change	Logical states kept; electrical levels rewritten
Factory reset	No reboot; defaults applied live
Short AP password (<8)	UI blocks; server ignores short non-empty (keeps previous)
Name > 15 chars	Server truncates with strncpy
Oversized schedule JSON	deserialize error → 400 Bad JSON
GPIO 1/3 as outputs	Breaks UART0 serial monitor after setup
STA disabled	NTP APIs fail gracefully; schedules use RTC
Manual + schedule both set	Manual wins until Auto

23. Libraries & Build Dependencies

Include	Source	Used for
WiFi.h	ESP32 core	STA + SoftAP
WiFiUdp.h	ESP32 core	NTP UDP packets
WebServer.h	ESP32 core	HTTP server :80
DNSServer.h	ESP32 core	Captive DNS :53
Preferences.h	ESP32 core	NVS key-value

Include	Source	Used for
ESPmDNS.h	ESP32 core	mDNS .local
Wire.h	Arduino/ESP32	I2C bus
ArduinoJson.h	Benoit Blanchon (lib)	REST JSON
RTCLib.h	Adafruit (lib)	DS3231 driver

Note: *ArduinoJson document sizes are chosen per handler (64 ... 8192). If you extend the API, watch stack/heap when enlarging StaticJsonDocument.*

24. Complete Function Index

Every function and method defined by the firmware (alphabetical groups).

Function / Method	Summary
timeHasElapsed	Unsigned millis interval test
isTimeReached	Unsigned millis deadline test
rtcDateTimeToUint64	RTCLib DateTime → uint64
uint64ToRtcDateTime	uint64 → DateTime (low 32)
gmtime64	UTC struct tm from uint64
localtime64	Local struct tm
getLocalEpoch	UTC + offsets
isActiveLow	Polarity with global override
getNTPInterval	Hours → ms
getRelayPin	Index → GPIO / -1
getActiveRelayCount	Active channel count
setRelayOutput	Logical state → pin level
initScheduleDefaults	Factory RelayConfig
setWiFiStationEnabled	Live STA on/off
initRTC	Probe DS3231 at boot
immediateDS3231Sync	Push epoch to DS3231
syncDS3231FromInternalRTC	Push without begin
performRTCReabase	Fold micros into epoch
getCurrentEpoch	Live UTC seconds
saveRTCState / loadRTCState	NVS epoch I/O
autoSaveInternalRTC	Hourly offline save
loadRTCFromDS3231	Chip → software clock
pauseWiFiForScan	Temp STA disconnect
calculateCriticalChecksum	Checkpoint integrity
SelfHealingSystem::saveCriticalState	NVS manual snapshot
SelfHealingSystem::restoreCriticalState	Boot manual restore
SelfHealingSystem::liveReconfigureWiFi	Soft WiFi repair
SelfHealingSystem::liveReconfigureMDNS	mDNS repair/announce
SelfHealingSystem::liveReconfigureDNS	DNS tick stub
SelfHealingSystem::liveReconfigureWebServer	Web tick stub
SelfHealingSystem::liveReconfigureAP	AP down recovery
SelfHealingSystem::restartAPIfNeeded	Forced AP bounce
SelfHealingSystem::recoverWiFi/MDNS/DNS/WebServer	Recovery wrappers
SelfHealingSystem::recoverNTP	Pool-wide NTP recover
SelfHealingSystem::recoverRTC	I2C RTC recover
SelfHealingSystem::performTargetedRecovery	Ordered full repair
SelfHealingSystem::verifyRelayStates	Pin level audit
SelfHealingSystem::smartRecovery	10 s health tick
performMemoryCleanup	Heap/scan hygiene
cleanupStateResources	Cache + scan cleanup
checkWebServerHealth	Idle web touch
checkAndCleanMemory	Heap monitor
checkBootButton	5 s hardware factory reset
loadGPIOConfig / saveGPIOConfig	GPIO NVS
syncInternalRTC	Commit new epoch everywhere

Function / Method	Summary
getNTP64Time	Blocking UDP NTP query
tryNTPSync	Throttled pool sync
handleBrowserTimeSync	API browser clock
beginWiFiConnect	Start STA attempt
updateRelayOutputs	Bulk pin apply helper
updateScheduleCache	Recompute schedule flags
processRelaySchedules	Engine + debounce drive
restartAP	AP restart wrapper
startMDNS / stopMDNS / restartMDNS	mDNS lifecycle
scheduleMDNSRestart	Deferred mDNS restart
getMDNSHostname / setMDNSHostname	Hostname accessors
setup	Boot sequence
loop	Main runtime
initDefaults	Factory settings writer
loadConfiguration / saveConfiguration	sys+relays NVS
loadExtConfig / saveExtConfig	extended NVS
setupWebServer	Routes + begin
handleGetRelays	GET relays
handleManualControl	POST manual
handleResetManual	POST auto
handleSaveRelay	POST schedules
handleRelayName	POST name
handleGetTime	GET time tick
handleGetWiFi / handleSaveWiFi	WiFi API
handleWiFiScanStart / handleWiFiScanResults	Scan API
handleGetNTP / handleSaveNTP / handleSyncNTP	NTP API
handleGetAP / handleSaveAP	AP API
handleGetSystem	Diagnostics
handleReset / handleFactoryReset	Maintenance
handleGetGPIOConfig / handleSaveGPIOConfig	GPIO map
handleAddGPIO / handleDeleteGPIO	GPIO mutate
handleToggleActiveLow / handleGlobalActiveMode	Polarity API

25. Quick Reference Cards

25.1 First Connect

1. Power ESP32
2. Join WiFi: ESP32_16CH_Timer_Switch / 12345678
3. Browse <http://192.168.4.1/>
4. GPIO → verify pins/polarity
5. WiFi → enable STA → scan → save network
6. Time → set offset → Sync NTP (or Browser)
7. Relays → name + schedules → Save

25.2 Schedule Cheat Sheet

start < stop → same-day window [start, stop)
 start > stop → overnight (uses yesterday bit after midnight)
 start = stop → always ON that day (if DOW matches)
 monthDays=0 → any day of month
 monthMask=0 → all months
 Multiple slots → OR (any match turns relay ON)
 Manual override → schedules ignored until Auto

25.3 Factory Reset

WEB: System → Factory Reset → confirm
 HW: Hold BOOT button 5 seconds
 RESULT: defaults restored, relays OFF, no reboot
 REJOIN: ESP32_16CH_Timer_Switch / 12345678

25.4 Time Source Dots (Header)

Dot colour	Meaning
Green (time)	NTP locked
Blue (time)	Browser or DS3231/RTC
Yellow (time)	No valid source yet
Green (wifi)	STA connected
Red (wifi)	STA not connected

25.5 Important URLs

URL	Page
<a href="http://<ip>/">http://<ip>/	Relays
<a href="http://<ip>/wifi">http://<ip>/wifi	WiFi Station
<a href="http://<ip>/ntp">http://<ip>/ntp	Time & NTP
<a href="http://<ip>/ap">http://<ip>/ap	Access Point
<a href="http://<ip>/gpio">http://<ip>/gpio	GPIO Config
<a href="http://<ip>/system">http://<ip>/system	System & Reset
<a href="http://<host>.local/">http://<host>.local/	mDNS entry (if supported)

25.6 Polarity Quick Set

Most relay modules (opto, active low coils): leave defaults
 → Global mode Per-Relay or Global Active LOW
 Active-high modules:
 → GPIO page → Global: All Relays Active HIGH
 Mixed bank:
 → Global Per-Relay → toggle each channel LOW/HIGH

End of Complete Features & Documentation.

This manual catalogues every feature, setting, API endpoint, UI page, schedule rule, time source, recovery path, NVS key, default value, and public function in the ESP32 16-Channel Automatic Relay Timer Switch firmware (config v11) with no

intentional omissions.

Author: **Raff Alds** · GitHub: github.com/xiv3r · License: **GPLv3**
Source file: `esp32-final-sketch.ino` (~3,982 lines) · Generated documentation